

SMART PAPER MILL PRODUCTION MONITORING DASHBOARD

A PROJECT REPORT

Submitted by

Manreet Gill (UID: 24BCS10479)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Industrial Training carried out at

SATIA INDUSTRIES LIMITED

Chandigarh University

June 2026

TRAINING COMPLETION CERTIFICATE

Certified that this project report "SMART PAPER MILL PRODUCTION MONITORING DASHBOARD" is the bonafide training work of Manreet Gill (UID: 24BCS10479) who carried out the project work under my/our supervision during the industrial training from 25 May 2026 to 10 July 2026.

<<Signature of the Supervisor>>

SIGNATURE

Mr. Surinder Mehra

SUPERVISOR

IT Head, Satia Industries Ltd.

Department of CSE, Chandigarh University

ACKNOWLEDGEMENT

I express my sincere gratitude to Satia Industries Limited for providing the opportunity to undertake my industrial training and to gain practical exposure to the production processes of a modern paper manufacturing plant. I am thankful to my industry supervisor and to the faculty of the Department of Computer Science & Engineering, Chandigarh University, for their continued guidance, encouragement and valuable feedback throughout the course of this project. I also acknowledge the support of my family and friends. The data used in this report is fictional and industry-realistic; no confidential or proprietary company data has been used.

TABLE OF CONTENTS

ABSTRACT	viii
ABBREVIATIONS	ix
INTRODUCTION	1
1.1 Client Identification / Need Identification / Relevant Contemporary Issue	1
1.2 Identification of Problem	1
1.3 Identification of Tasks	2
1.4 Timeline	2
1.5 Organization of the Report	3
LITERATURE REVIEW / BACKGROUND STUDY	4
2.1 Timeline of the Reported Problem	4
2.2 Proposed Solutions	4
2.3 Bibliometric Analysis	4
2.4 Review Summary	5
2.5 Problem Definition	5
2.6 Goals / Objectives	5
DESIGN FLOW / PROCESS	6
3.1 Evaluation & Selection of Specifications / Features	6
3.2 Design Constraints	6
3.3 Analysis and Feature Finalization subject to Constraints	6
3.4 Design Flow	6
3.5 Design Selection	7
3.6 Implementation Plan / Methodology	7
RESULTS ANALYSIS AND VALIDATION	10
4.1 Implementation of Solution	10
4.2 Testing / Characterization / Data Validation	11
CONCLUSION AND FUTURE WORK	13

5.1 Conclusion	13
5.2 Future Work.....	13
REFERENCES	14
APPENDIX	15
Appendix 1: Demonstration Login Credentials.....	15
Appendix 2: Database Schema (Principal Tables)	15
Appendix 3: Principal REST API Endpoints	15
USER MANUAL	16
Step 1: Install dependencies (once)	16
Step 2: Start the application.....	16
Step 3: Open the application.....	16
Step 4: Sign in.....	16
Step 5: Use the modules	16
Step 6: Sign out.....	17

LIST OF FIGURES

Figure : Project timeline (ten-week internship schedule)	2
Figure : Three-tier system architecture	8
Figure : Entity-relationship diagram of the database schema	8

LIST OF TABLES

Table : List of abbreviations used in the report	ix
Table : Identification of project tasks	2
Table : Comparison of existing solution classes.....	4
Table : Comparison of two candidate designs	6
Table : Technology stack.....	8
Table : Implemented modules	10
Table : Summary of automated test results.....	11
Table : Requirement completion against the SRS	11
Table : Role-based demonstration accounts	15

ABSTRACT

This report presents the design, development and validation of the Smart Paper Mill Production Monitoring Dashboard, a web-based application developed during industrial training at Satia Industries Limited, a manufacturer of writing and printing paper from agricultural residue (wheat straw) and wood pulp. The objective of the project is to replace fragmented, paper-based production logging with a single, real-time, role-based digital platform that consolidates production output, machine status, quality parameters, downtime and Overall Equipment Effectiveness (OEE). The system is implemented as a Python (Flask) backend with an SQLite database, secured using bcrypt password hashing and JSON Web Token (JWT) authentication with role-based access control, and a responsive HTML5/CSS3/JavaScript front-end that uses Chart.js for visualization. Live machine telemetry is delivered to the browser using Server-Sent Events, demonstrating real-time monitoring without page reloads. Because live plant-floor integration is outside the scope of an academic assignment, the system operates on realistic, internally consistent simulated data. The delivered solution includes secure login, an administrative panel for users, machines and thresholds, production and downtime data entry, automatic OEE computation, threshold-based alerts, analytical reports with CSV and PDF export, an audit trail, and a dark-mode interface. The backend was validated using an automated test suite of ninety-four test cases, all of which pass. The project demonstrates the core principles of industrial production monitoring while remaining free, lightweight and runnable on standard student hardware.

ABBREVIATIONS

Table 1: List of abbreviations used in the report

Abbreviation	Expansion
OEE	Overall Equipment Effectiveness
GSM	Grams per Square Metre (paper grammage)
TPD	Tonnes Per Day
JWT	JSON Web Token
RBAC	Role-Based Access Control
SSE	Server-Sent Events
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
KPI	Key Performance Indicator
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
PLC / DCS	Programmable Logic Controller / Distributed Control System

CHAPTER 1

INTRODUCTION

1.1 Client Identification / Need Identification / Relevant Contemporary Issue

Satia Industries Limited is an established manufacturer of writing and printing paper that uses agricultural residue, principally wheat straw, together with wood pulp as raw material. Paper production is a continuous, multi-stage process spanning raw-material handling, pulping, chemical processing and bleaching, the paper machine section, drying, finishing and cutting, packaging and dispatch. The performance of each stage directly affects output, product quality and operating cost.

At present, much of the production information at the plant is recorded manually in shift registers, logbooks and disconnected spreadsheets maintained separately for each section. This practice is consistent with a wider industry challenge: as the manufacturing sector moves towards Industry 4.0 and digitalisation, plants that still rely on manual record-keeping lack the real-time visibility required for timely decisions. The need for a consolidated monitoring tool was identified through observation of the plant's record-keeping practices during the training period.

The relevant contemporary issue is the digital transformation of process industries and, in particular, the use of Overall Equipment Effectiveness as a standard metric for measuring manufacturing productivity. A monitoring dashboard that brings these concepts to a paper mill addresses a genuine, documented operational need.

1.2 Identification of Problem

The broad problem addressed by this project is the absence of a single, real-time, role-based digital platform through which paper-mill production, quality and downtime information can be recorded, monitored and analysed in a consistent and timely manner. The consequences include delayed managerial decisions, laborious consolidation of scattered data, late detection of out-of-specification quality, and an inability to compare the performance of different shifts, machines or time periods.

1.3 Identification of Tasks

To build and validate the solution, the work was divided into the following tasks, which also form the structure of the report:

Table 2: Identification of project tasks

Task	Description
Requirements	Prepare the Software Requirements Specification and database design.
Authentication	Implement secure login, JWT and role-based access control.
Data entry	Build production, maintenance and downtime data-entry modules.
Real-time	Provide a live data feed (simulator) to the monitoring screens.
Dashboard	Develop KPI cards, machine-status board and trend charts.
OEE & alerts	Compute OEE and generate threshold-based alerts.
Reports	Build analytical reports with filters and CSV/PDF export.
Administration	Provide an admin panel for users, machines and thresholds.
Testing	Validate the backend with an automated test suite.

1.4 Timeline

The project was planned over a ten-week internship schedule. The timeline is shown as a Gantt chart in Figure 1, with each phase delivering a defined increment of functionality.

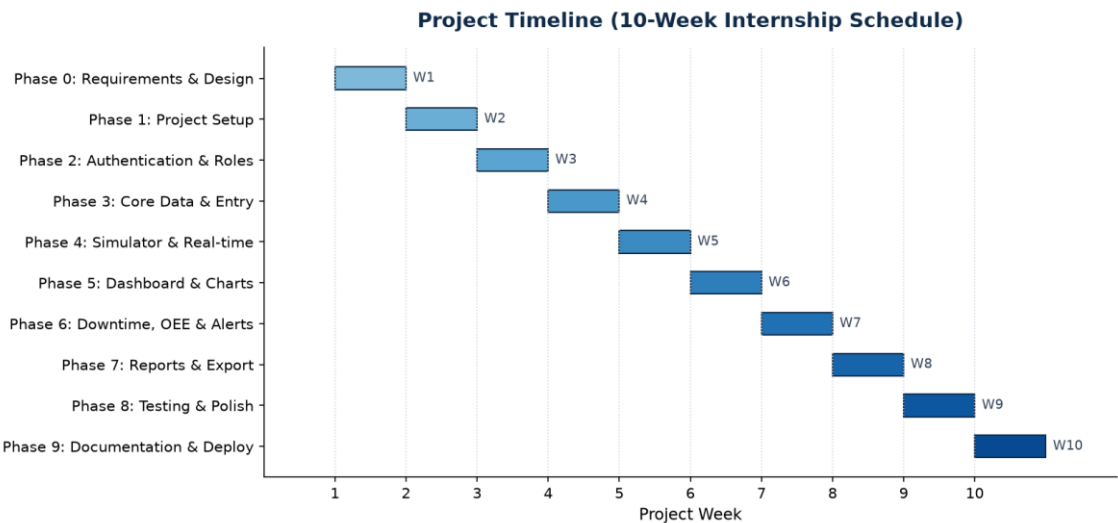


Figure 1: Project timeline (ten-week internship schedule)

1.5 Organization of the Report

The remainder of the report is organised as follows. Chapter 2 presents the literature review and background study, defines the problem and states the objectives. Chapter 3 describes the design flow, including the evaluation of features, the design constraints, two alternative designs and the selection of the final design. Chapter 4 presents the implementation using modern engineering tools together with the testing and validation results. Chapter 5 concludes the report and outlines future work, followed by the references, appendices and the user manual.

CHAPTER 2

LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

The problem of monitoring and improving manufacturing productivity has been studied for several decades. Overall Equipment Effectiveness was introduced by Seiichi Nakajima in the 1980s as part of Total Productive Maintenance and remains the standard composite metric, defined as the product of Availability, Performance and Quality. From the 1990s onwards, Manufacturing Execution Systems and Supervisory Control and Data Acquisition systems were increasingly deployed to collect plant data automatically. More recently, the Industry 4.0 movement has emphasised connected sensors, real-time dashboards and data-driven decision making across process industries, including pulp and paper.

2.2 Proposed Solutions

Several classes of solution to production-monitoring problems exist. Commercial MES and OEE software platforms offer comprehensive data collection and analytics but are costly and complex to deploy. SCADA systems provide real-time control-room visualisation but require integration with plant instrumentation through industrial protocols. Lightweight custom dashboards, built with open web technologies, offer a low-cost alternative suited to demonstration and small-scale deployments.

2.3 Bibliometric Analysis

Table 3: Comparison of existing solution classes

Solution class	Key features	Effectiveness	Drawback
Commercial MES	End-to-end data, analytics, ERP links	High	Expensive, complex
SCADA / DCS	Real-time control and visualisation	High	Needs plant instrumentation
Spreadsheets	Familiar, low cost	Low	Manual, no real-time, error-prone
Custom web dashboard	Lightweight, configurable, free	Medium	Requires development effort

2.4 Review Summary

The literature confirms that OEE-based monitoring is the accepted approach and that real-time dashboards deliver clear benefits. For an academic internship, where cost must be zero and the solution must run on standard hardware without plant access, a lightweight custom web dashboard is the most appropriate option. This project therefore implements such a dashboard, reproducing the essential concepts of industrial monitoring on realistic simulated data.

2.5 Problem Definition

The project shall design and implement a web-based dashboard that records and consolidates paper-mill production, quality and downtime data; computes OEE; visualises live machine status and historical trends; raises threshold-based alerts; and enforces role-based access. The system shall be free, lightweight and runnable on a standard laptop. It shall not require live plant instrumentation, and shall not use any confidential company data.

2.6 Goals / Objectives

1. Provide a centralized, role-based dashboard for real-time paper-mill production monitoring.
2. Capture production data, including grammage, moisture and machine speed, and flag out-of-specification readings.
3. Compute Overall Equipment Effectiveness automatically from availability, performance and quality.
4. Deliver live machine telemetry to the browser without manual page reloads.
5. Generate threshold-based alerts and present them in a dedicated panel.
6. Provide analytical reports with date, shift and machine filters and CSV/PDF export.
7. Provide an administrative panel for managing users, machines and thresholds, with an audit log.
8. Validate the backend with an automated test suite.

CHAPTER 3

DESIGN FLOW / PROCESS

3.1 Evaluation & Selection of Specifications / Features

Drawing on the literature review, the features ideally required in the solution were identified: secure role-based login; real-time key performance indicators; a live machine-status board; production and downtime data entry; automatic OEE calculation; threshold-based alerts; historical reports with export; and an administrative panel. These were prioritised into core features (essential for demonstration) and supporting features (value-adding), and recorded in the Software Requirements Specification.

3.2 Design Constraints

The design was developed under the following constraints. Economic: the solution must be built entirely with free and open-source tools, with no licensing or API cost. Manufacturability / deployability: it must run on standard student hardware and a normal web browser. Health, safety and environmental: the system is informational and introduces no physical risk; it reflects the plant's agro-residue, low-impact raw material. Professional and ethical: no confidential or proprietary company data is used, and all demonstration data is fictional. Social and regulatory: the interface is accessible and conveys status through icon, colour and label together rather than colour alone.

3.3 Analysis and Feature Finalization subject to Constraints

In light of these constraints, live sensor and PLC integration was removed from scope and replaced by a built-in data simulator and manual data entry. Predictive-maintenance and machine-learning features were deferred to future work. The remaining feature set was retained and finalised as the basis for implementation.

3.4 Design Flow

Two alternative designs were considered for the technology stack, as compared in Table 3.

Table 4: Comparison of two candidate designs

Aspect	Alternative A (Node stack)	Alternative B (Python
--------	----------------------------	-----------------------

		stack) — selected
Backend	Node.js + Express	Python + Flask
Database	MySQL (server required)	SQLite (serverless, built-in)
Real-time	Socket.IO (extra dependency)	Server-Sent Events (built-in)
Runtime availability	Requires Node runtime	Python widely pre-installed
Setup effort	Higher	Lower
Cost	Free	Free

3.5 Design Selection

Alternative B was selected. Although the project was originally specified around a Node.js stack, the Python/Flask stack with SQLite and Server-Sent Events was chosen because it requires no separate database server, no additional real-time library and no runtime that is not already commonly available, while remaining equally free. This minimises setup effort and maximises the likelihood that the application runs unmodified on a standard laptop, satisfying the deployability constraint.

3.6 Implementation Plan / Methodology

The system follows a three-tier architecture comprising a presentation layer (the web browser), an application layer (the Flask REST API, which also serves the static front-end) and a data layer (the SQLite database). Authentication uses bcrypt-hashed passwords and JWTs, and every protected route enforces a role check. The architecture is shown in Figure 2 and the database entity-relationship design in Figure 3.

System Architecture — Smart Paper Mill Production Monitoring Dashboard

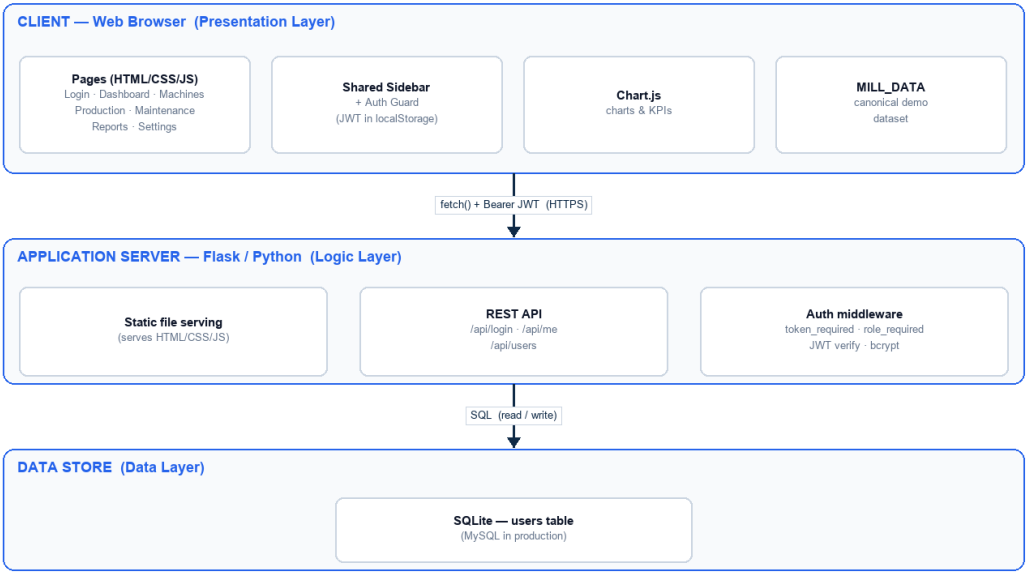


Figure 2: Three-tier system architecture

Entity-Relationship Diagram — Smart Paper Mill Production Monitoring Dashboard

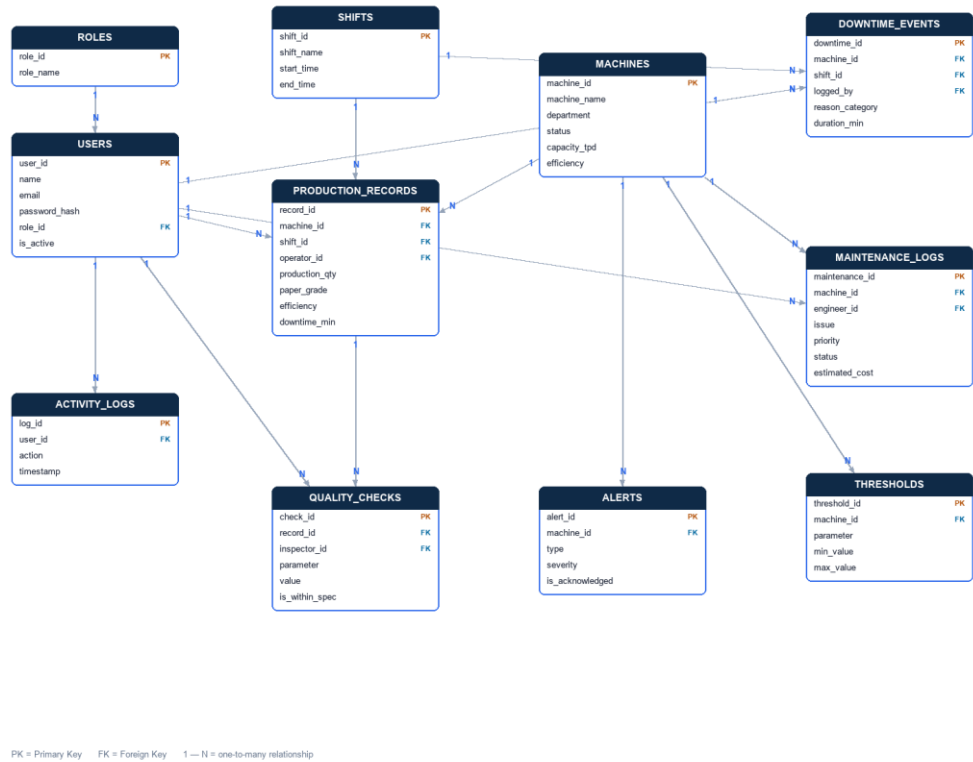


Figure 3: Entity-relationship diagram of the database schema

The technologies used are summarised in Table 4.

Table 5: Technology stack

Layer	Technology	Purpose
-------	------------	---------

Front-end	HTML5, CSS3, JavaScript	Structure, styling, interactivity
Charts	Chart.js	KPI and trend visualisation
Backend	Python (Flask)	REST API and static file serving
Database	SQLite	Persistent storage
Security	bcrypt, PyJWT	Password hashing and token authentication
Real-time	Server-Sent Events	Live telemetry push
Testing	pytest	Automated backend tests
Version control	Git	Source management

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The solution was implemented using modern engineering tools: Visual Studio Code as the editor; Python with the Flask framework and SQLite for the backend; HTML, CSS, JavaScript and Chart.js for the front-end; Git for version control; and pytest for automated testing. The supplementary figures and the report itself were generated programmatically using Python libraries (matplotlib and python-docx). The delivered modules are listed in Table 5.

Table 6: Implemented modules

Module	Function
Authentication	JWT login, role-based access, session handling
Dashboard	KPI cards (incl. average OEE), trend and status charts, live efficiency feed
Machine Monitoring	Status board with live temperature and efficiency via SSE
Production Entry	Records output, GSM, moisture and speed; flags out-of-spec readings
Downtime Logging	Logs events with reason and automatically computed duration
Maintenance	Logs and tracks maintenance jobs by priority and status
Reports	Filterable charts, OEE by machine, CSV and PDF export
Alerts	Threshold-driven alert panel with acknowledgement
Admin Panel	Manage users, machines and thresholds; view the audit log

Key engineering results include the automatic OEE computation (Availability x Performance x Quality) derived from production records and machine capacity; live machine telemetry pushed from the backend every three seconds; persistent storage of all production, downtime, maintenance, user, machine and threshold data; and an audit trail recording logins and key changes.

4.2 Testing / Characterization / Data Validation

The backend was validated using an automated test suite executed with pytest, using an isolated temporary database for each test. All ninety-four test cases pass — a core API suite of thirty-six tests and a role-workflow suite of fifty-eight tests that exercise each role's tasks end to end. The categories of tests are summarised in Table 6.

Table 7: Summary of automated test results

Test suite / category	Cases	Result
Core API — health & authentication	5	Pass
Core API — role-based access control	2	Pass
Core API — production records & quality	6	Pass
Core API — downtime events	4	Pass
Core API — maintenance logs	2	Pass
Core API — live-stream authentication	2	Pass
Core API — user management	5	Pass
Core API — machine management	5	Pass
Core API — thresholds	3	Pass
Core API — activity / audit log	2	Pass
Role-workflow scenarios (Admin / Manager / Supervisor / Operator)	58	Pass
Total	94	All Pass

In addition to automated tests, the running application was validated through live request tests that confirmed the live telemetry stream, role enforcement, data persistence across restarts and the correct computation of downtime duration. Against the Software Requirements Specification, the functional requirements are approximately 88% complete, the non-functional requirements approximately 81%, and the feature set approximately 87%, as summarised in Table 7.

Table 8: Requirement completion against the SRS

Section	Completion
Functional Requirements	~88%
Non-Functional Requirements	~81%
Features	~87%
User Roles (four implemented)	~88%

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The industrial training project successfully delivered a functional, web-based dashboard for monitoring the production activities of a paper manufacturing plant. The system consolidates production, quality, downtime, maintenance and machine information within a single, secure, role-based interface; computes Overall Equipment Effectiveness; delivers live telemetry without page reloads; raises threshold-based alerts; provides analytical reports with export; and includes a complete administrative panel and audit trail. The backend is validated by a ninety-four-case automated test suite, all passing.

The principal deviation from the original specification is the absence of live plant-floor instrumentation. This was a deliberate scope decision: integrating with real sensors and PLCs requires industrial protocols and plant access that are not available in an academic setting. Live data was therefore replaced by an internally consistent simulator, which fully demonstrates the monitoring concepts. The original Node.js technology choice was also revised to a Python/Flask stack for the deployability reasons discussed in Chapter 3.

5.2 Future Work

The solution provides a clear path for further development. The recommended next steps are:

- Integration with real sensors and PLCs using industrial protocols such as OPC-UA or Modbus, replacing the simulator with live measurements.
- Predictive maintenance using statistical or machine-learning techniques to anticipate equipment failure.
- Migration of the database from SQLite to MySQL and deployment to a free cloud tier for a live demonstration.
- Addition of a dedicated quality-inspection module and a read-only viewer role.
- A native or progressive mobile application for supervisors on the move.
- Automated report scheduling and notification by electronic mail.

REFERENCES

- Chart.js (2024) "Chart.js Documentation", <https://www.chartjs.org> (accessed 2026).
- Flask (2024) "Flask Documentation (3.x)", Pallets Projects, <https://flask.palletsprojects.com> (accessed 2026).
- Grinberg, M. (2018) "Flask Web Development", 2nd ed., O'Reilly Media, Sebastopol, CA.
- Jones, M., Bradley, J. and Sakimura, N. (2015) "JSON Web Token (JWT)", RFC 7519, Internet Engineering Task Force.
- Nakajima, S. (1988) "Introduction to TPM: Total Productive Maintenance", Productivity Press, Cambridge, MA.
- Provos, N. and Mazieres, D. (1999) "A Future-Adaptable Password Scheme", Proc. USENIX Annual Technical Conference, pp. 81-91.
- SQLite (2024) "SQLite Documentation", <https://www.sqlite.org> (accessed 2026).

APPENDIX

Appendix 1: Demonstration Login Credentials

Table 9: Role-based demonstration accounts

Role	Username	Password
Administrator	admin	Admin@123
Plant Manager	manager	Manager@123
Shift Supervisor	supervisor	Super@123
Machine Operator	operator	Operator@123

Appendix 2: Database Schema (Principal Tables)

- users — accounts with bcrypt password hash, role and active flag.
- production_records — date, shift, machine, operator, quantity, grade, GSM, moisture, speed.
- maintenance_logs — machine, problem, priority, engineer, date, status.
- downtime_events — date, machine, shift, reason, start, end, duration.
- machines — machine ID, name, department, capacity, status, active flag.
- thresholds — configurable limits (GSM, moisture, temperature, efficiency, downtime).
- activity_logs — audit trail of logins and key changes.

Appendix 3: Principal REST API Endpoints

- POST /api/login, GET /api/me — authentication.
- GET/POST/PUT/DELETE /api/users — user management (Administrator).
- GET/POST/PUT/DELETE /api/machines — machine management (Administrator).
- GET/PUT /api/thresholds — threshold configuration.
- GET/POST/PUT/DELETE /api/production — production records.
- GET/POST/PUT/DELETE /api/downtime — downtime events.
- GET/POST/PUT/DELETE /api/maintenance — maintenance logs.
- GET /api/stream — live telemetry (Server-Sent Events).
- GET /api/activity — audit log (Administrator).

USER MANUAL

Step 1: Install dependencies (once)

Open a terminal in the project folder and run: `pip install -r backend/requirements.txt`

Step 2: Start the application

Run: `python backend/app.py`. The terminal confirms that the server is running at `http://localhost:5000/`.

Step 3: Open the application

Open a web browser and navigate to `http://localhost:5000`. The login page is displayed.

Step 4: Sign in

Sign in using the credentials for your role (see Appendix 1). For full access, sign in as Administrator with username 'admin' and password 'Admin@123'. On success, the Dashboard opens.

Step 5: Use the modules

- Dashboard — view KPI cards, charts and the live efficiency indicator.
- Machines — monitor live machine status, temperature and efficiency.
- Production — add production records, including GSM, moisture and speed; out-of-spec values are flagged.
- Downtime — log machine stoppages; the duration is computed automatically.
- Maintenance — log and track maintenance jobs.
- Reports — filter by period, machine and shift; export as CSV or PDF.
- Alerts — review and acknowledge threshold-based alerts.
- Settings (Administrator) — manage users, machines and thresholds, and view the activity log; toggle dark mode.

Step 6: Sign out

Use the Logout option at the bottom of the sidebar to end the session and return to the login page. Detailed, role-specific user manuals are provided separately in the project documentation.